UDP vs TCP

when speed beats reliability

Watch the 3-min video

In one line: UDP sends independent datagrams with no handshake, ACKs, retransmission or ordering; choose it when fresh data beats complete data (voice, video, games), for tiny exchanges like DNS, or as the base for protocols like QUIC, and choose TCP when losing a byte would be a bug.

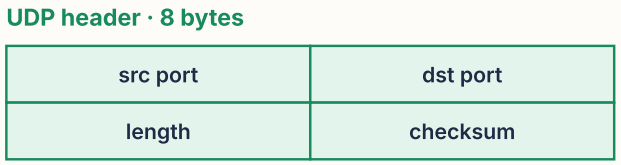
1 UDP: the simplest transport

You are on a video call and the connection hiccups for a split second. Do you want the call to freeze while every lost packet is resent? No. You would rather lose a few milliseconds of audio and keep talking. That is exactly what UDP gives you.

UDP sends independent messages called **datagrams**. There is no handshake, no connection, no acknowledgements, no retransmission and no ordering. Each datagram may arrive, arrive late, arrive out of order, arrive twice, or not arrive at all. Unlike TCP's byte stream, each datagram keeps its boundaries: one send is one receive.

2 What you get in return

- **Tiny overhead.** The UDP header is just **8 bytes**, against at least 20 for TCP.
- **No setup round trip.** The very first packet carries data.
- **No head-of-line blocking.** A lost datagram never holds up the next one.
- **No per-connection state** on the server, so one socket can serve huge numbers of clients.
- **Multicast and broadcast.** One packet to many receivers, which TCP cannot do.



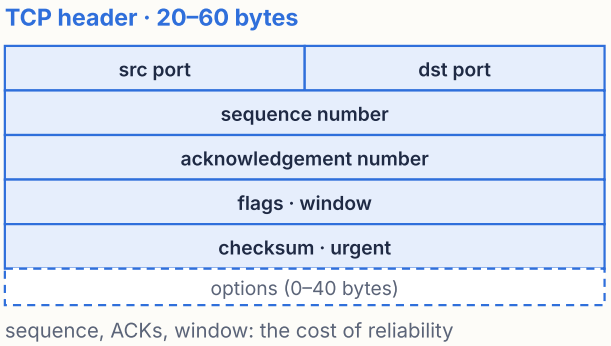


Figure 1. UDP adds only ports, a length and a checksum to IP. TCP's extra fields are what make reliability, ordering and flow control possible.



Figure 2. TCP waits a round trip before sending and holds later data behind a loss. UDP sends at once and hands every datagram that arrives straight to the app.

3 Real-time voice: why resending is pointless

Voice apps send audio in small packets, about one every **20 ms** (50 packets per second). If one is lost, resending it is pointless: by the time it arrives, at least one more round trip later, that moment of the conversation has already played.

So voice apps use UDP, keep a small **jitter buffer** (a few tens of milliseconds) to smooth out uneven arrival times, and simply **hide the gap** by repeating or blending the neighbouring audio. Losing 1–2% of 20 ms packets is barely noticeable; freezing for 150 ms on every loss is not.



Figure 3. A voice stream over UDP. The lost packet is concealed, not resent; the jitter buffer absorbs small variations in arrival time.

4 Choosing between them

Choose UDP when...	Choose TCP when...
Freshness beats completeness: voice and video calls, live broadcasts, online games	**Every byte must arrive, in order**: web pages and APIs, file transfers, database connections, payments
Tiny request/response: DNS (one packet out, one back), NTP, DHCP, metrics (StatsD)	**Large or long transfers** where you want congestion control and flow control for free
Building a new protocol with its own reliability: QUIC (HTTP/3), WebRTC media	**Losing data would be a bug**, and you don't want to rebuild reliability yourself

THE RULE OF THUMB

If losing data would be a bug, use TCP. If late data is worthless, use UDP. If you need reliability *and* UDP's freedoms (no head-of-line blocking, faster setup), use a protocol built on UDP such as QUIC rather than writing your own.

5 How a game handles lost UDP packets

A multiplayer game server sends the **whole relevant game state** many times per second (often 20–60 updates per second, up to 128 in competitive shooters). If one update is lost, the next one replaces it anyway, so there is nothing to resend.

The client **predicts** movement (its own player reacts instantly, other players are extrapolated) and **smooths** between updates, so a missing update is invisible. For the few things that must arrive, such as a purchase, a chat message or a match result, the game adds **its own acknowledgements and resends**, just for those messages, usually with an ID so a duplicate is ignored.

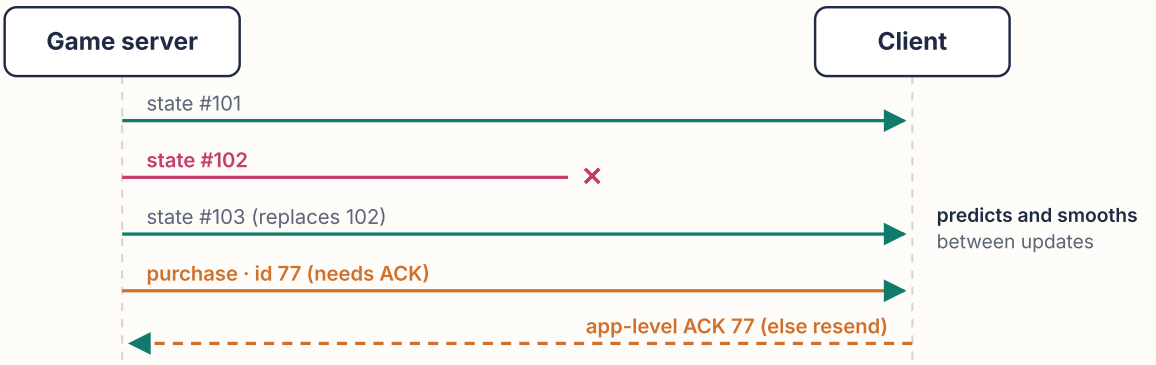


Figure 4. State updates are fire-and-forget: update 103 replaces the lost 102. Only the purchase is tracked with an application-level ACK and resent if needed.

WORKED EXAMPLE: WHAT DOES A LOSS COST?

Voice at 20 ms per packet, 150 ms round trip, 1% loss.

- **TCP:** each loss stalls the stream for at least one RTT plus detection time, about **150–300 ms** of silence, then a burst of stale audio. At 50 packets/s, that is roughly one stall every 2 seconds.
- **UDP:** each loss is a **20 ms** gap, concealed by the decoder. Total delay stays at the network delay plus a ~40 ms jitter buffer.

COMMON MISTAKES

- Saying UDP is “faster” in general. It is faster to start and never stalls on loss, but the bytes travel at the same speed.
- Sending large UDP payloads. Anything bigger than the path MTU (about 1,200–1,400 bytes to be safe) gets fragmented, and losing one fragment loses the whole datagram.
- Forgetting congestion: UDP has no congestion control, so a careless sender can flood a network. Rate-limit it yourself.
- Reinventing TCP on UDP for an API. Use TCP, or QUIC if you need what UDP offers.

6 Key terms

Datagram: one self-contained UDP message

Jitter buffer: short delay that evens out arrival

Tick rate: game state updates per second

Connectionless: no handshake, no shared state

Loss concealment: hiding a missing audio or video frame

Client prediction: simulating movement between updates

Jitter: variation in packet arrival times

Multicast: one packet delivered to many receivers

QUIC: reliable, encrypted transport on UDP

7 Interview questions

Q1. When would you choose UDP over TCP? BASIC

When low latency matters more than completeness (voice, video, games), for tiny exchanges like DNS where one packet each way is enough, or as the base for a protocol like QUIC that adds its own reliability. Otherwise, TCP.

Q2. What does UDP give up compared with TCP, and what does it gain? BASIC

It gives up the handshake, ACKs, retransmission, ordering, flow and congestion control. It gains an 8-byte header (vs 20+), no setup round trip, no head-of-line blocking, message boundaries and multicast.

Q3. How does a game handle lost UDP packets? BASIC

It sends frequent full state updates, so the next update replaces a lost one; the client predicts and smooths between updates; and only critical events (purchases, chat) get app-level ACKs and resends, with IDs to drop duplicates.

Q4. You're designing a video-calling service. TCP or UDP for the media, and what about signalling? SCENARIO

Media over UDP (RTP via WebRTC) with a jitter buffer and loss concealment, since late frames are worthless. Signalling (join, leave, chat) over a reliable channel such as WebSocket over TCP. Fall back to TURN over TCP or TLS only when UDP is blocked.

Q5. Your service pushes metrics to a collector 10,000 times a second. Would UDP be OK? SCENARIO

Yes, StatsD does exactly this: fire-and-forget over UDP, so a slow collector never blocks the app. Losing 0.1% of samples barely moves an aggregate. For billing-grade counts, use TCP or a queue instead.

Q6. Why does DNS use UDP, and when does it switch to TCP? SCENARIO

A query and its answer usually fit in one packet each, so UDP avoids a handshake and server state. If the answer is too large (truncated flag set), or for zone transfers, it retries over TCP.

Q7. If UDP is unreliable, why is HTTP/3 built on it? TRICKY

QUIC adds reliability, congestion control and TLS 1.3 on top of UDP, but tracks each stream separately, so one loss doesn't stall other streams. It also combines the transport and TLS handshakes into one round trip, and it runs in user space, so it can evolve without OS kernel changes.

Q8. A client sends one 6 KB UDP datagram and it often never arrives, while 1 KB datagrams do. Why? TRICKY

6 KB exceeds the typical 1,500-byte MTU, so IP fragments it into about 5 pieces. Lose any one and the whole datagram is dropped, and some firewalls drop fragments outright. Keep UDP payloads under about 1,200 bytes.

Next: SD-B-006 · IP Addresses & CIDR backendroadmap.com